Índice

1. Resumen ejecutivo y dictamen 1

2. PAT expuesto: estado y procedimiento de revocación 2

3. Rotación coordinada de secretos 3

3.1 INTERNAL_API_SECRET — matriz de consumo 3

3.2 Procedimiento de rotación de INTERNAL_API_SECRET 3

3.3 APP_ENCRYPTION_KEY — matriz y procedimiento 4

4. Credenciales en artefactos locales e historial 5

5. Verificación de las ramas remotas publicadas 6

6. Plan de despliegue a producción 6

6.1 Variables y dependencias por servicio 7

6.2 Orden de arranque y comprobaciones 8

6.3 Validación con WhatsApp controlado 8

6.4 Autenticación, persistencia y recuperación 9

6.5 Rollback 9

7. Criterios para declarar "Producción Verificada" 9

8. Acciones del propietario y anexo operativo 10

Anexo: verificación rápida post-despliegue 11

1. Resumen ejecutivo y dictamen

El ecosistema ALBRA quedó publicado el 27 de octubre de 2026 en sus tres repositorios (ALBRA8/CRM-ALBRA, ALBRA8/WHATSPOWER y ALBRA8/WHATSPOWER-ULTRA) mediante una publicación selectiva que aisló los commits de misión del trabajo local ajeno, sin force push y sin tocar ramas remotas ajenas. Esta fase final de cierre no rehace integraciones ni repite auditorías anteriores: su alcance fue verificar el estado post-publicación, preparar la rotación de los secretos compartidos, confirmar la higiene de credenciales y dejar operativo el plan de despliegue a producción con criterios de aceptación medibles.

Todas las comprobaciones de esta fase se ejecutaron sin reutilizar el token expuesto: la verificación de ramas remotas se hizo con acceso anónimo de lectura, el escaneo de credenciales corrió sobre artefactos locales y el plan de despliegue se construyó a partir del código publicado y de la guía DEPLOY.md del propio repositorio. No se habilitaron campañas ni envíos reales; los interruptores ALLOW_REAL_DISPATCH permanecen en false en WHATSPOWER y WHATSPOWER-ULTRA.

3 / 3
SHAs publicados verificados en origin/main

0
credenciales reales en artefactos locales

4
procesos comparten INTERNAL_API_SECRET

Tarea del encargo	Estado	Evidencia principal
1. Confirmar revocación del PAT expuesto	**PENDIENTE** (propietario)	No reutilizable desde esta sesión; procedimiento en capítulo 2
2. Preparar rotación de INTERNAL_API_SECRET y APP_ENCRYPTION_KEY	**COMPLETADO** (preparación)	Matriz de consumo y procedimiento en capítulo 3
3. Comprobar ausencia de credenciales reales	**COMPLETADO**	Escaneo final: 525 archivos, 0 reales (capítulo 4)
4. Confirmar commits correctos en ramas remotas	**COMPLETADO**	ls-remote anónimo: SHAs exactos (capítulo 5)
5. Plan de despliegue a producción	**COMPLETADO**	Capítulo 6 (variables, arranque, validación, rollback)
6. No habilitar campañas ni mensajes reales	**VIGENTE**	ALLOW_REAL_DISPATCH=false; sin envíos en esta fase
7. No declarar producción verificada	**VIGENTE**	Criterios P-01..P-08 pendientes en entorno real (capítulo 7)

Tabla 1. Estado de las siete tareas del encargo de cierre.

Dictamen: el sistema está en condiciones de iniciar un despliegue controlado a producción, pero existen tres dependencias exclusivas del propietario que lo bloquean: la revocación del PAT expuesto, la rotación

de la clave NVIDIA (que reside en el historial publicado) y la rotación efectiva de los dos secretos coordinados. Además, la declaración de "Producción Verificada" queda supeditada a la ejecución documentada de los criterios del capítulo 7 en el entorno real; nada de lo verificado hasta aquí sustituye esa batería final.

2. PAT expuesto: estado y procedimiento de revocación

Durante la fase de publicación, un personal access token de GitHub se pegó en el chat para posibilitar los pushes; ese acto lo convirtió en credencial expuesta. El último registro de trabajo lo daba por **activo**, con revocación pendiente del propietario. En esta sesión el token no fue reutilizado ni reconstruido: ninguna operación requirió autenticación (los tres repositorios son legibles de forma anónima) y ninguna herramienta lo almacenó. La confirmación de que ya está inválido no puede obtenerse desde aquí sin enviarlo de nuevo, precisamente lo que la política de seguridad prohíbe, de modo que la verificación se delega en el propietario con el procedimiento siguiente.

Procedimiento de revocación y verificación (propietario):

- Abrir github.com/settings/tokens (Personal access tokens) y localizar el token clásico con prefijo ghp_ usado en la publicación.
- Pulsar Revoke/Delete. Los tokens clásicos también caducan solos si tienen expiración; no debe esperarse a esa fecha.
- Verificar la revocación sin pegarlo en ningún chat: desde una terminal del propietario, una llamada curl con el token debe devolver HTTP 401 tras la revocación.
- Revisar Security log de la cuenta (github.com/settings/security-log) en busca de usos anómalos entre la exposición y la revocación.
- Repasar tokens antiguos de la misma org (aparecen en el historial de worklog.md) y revocarlos también.

Un dato agravante documentado ya en DEPLOY.md (sección 12): el historial publicado de main contiene una clave NVIDIA completa (prefijo nvapi-) y PATs ghp_ en commits históricos del worklog. Mientras no se reescriba el historial, la única mitigación válida es revocar/rotar esas credenciales en sus plataformas de origen; borrarlas de los archivos nuevos no las retira de los commits antiguos. La reescritura con git filter-repo queda clasificada como mejora opcional (P2) porque la rotación neutraliza el riesgo operativo.

```
# Verificacion post-revocacion (ejecutarla el propietario, fuera de chats):
curl -s -o /dev/null -w "%{http_code}" -H "Authorization: Bearer <TOKEN>" \
https://api.github.com/user
# Resultado esperado tras revocar: 401
```

3. Rotación coordinada de secretos

Los dos secretos del encargo no son independientes: INTERNAL_API_SECRET es un secreto **compartido entre cuatro procesos** que deben actualizarse en la misma ventana, mientras que APP_ENCRYPTION_KEY protege datos en reposo y exige re-cifrado de los valores ya almacenados. Esta capítulo identifica consumidores exactos, procedimiento de actualización y verificación por servicio, sin imprimir ningún valor.

3.1 INTERNAL_API_SECRET — matriz de consumo

Proceso	Puerto	Cómo lo usa	Detalle verificado en código
CRM app (Next.js)	3000	Sale como Bearer al daemon; exige X-Internal-Secret en rutas internas	daemon-proxy (Bearer saliente); auto-reply, transcribe, handoff y describe-image validan X-Internal-Secret; inbox.ts y workflow-engine.ts llaman al daemon; /api/doctor reporta solo su longitud
wa-daemon (Baileys)	3002	Valida Authorization: Bearer en cada endpoint	Arranque fail-closed: sin valor de 24+ caracteres el daemon no arranca; devuelve 401 Unauthorized sin Bearer válido
WHATSPOWER	3003	Sale como Bearer al daemon y valida Bearer entrante	src/server.ts usa INTERNAL_API_SECRET para /send y para dos rutas internas; **incluye un fallback por defecto en el código publicado** (ver nota)
WHATSPOWER-ULTRA	3004	Sale como Bearer al dispatcher del daemon	src/adapters/dispatcher.ts envía Bearer a {WA_DAEMON_URL}/send tomando INTERNAL_API_SECRET del entorno

Tabla 2. Consumidores de INTERNAL_API_SECRET confirmados en el código de los tres repos.

Nota de hallazgo: en WHATSPOWER publicado (commit db0d088), si la variable no está definida el servidor adopta un valor por defecto escrito en el propio código, con lo que un despliegue descuidado autenticaría con un secreto público. El arranque correcto debe fijar siempre la variable; eliminar ese fallback es un cambio de código recomendado para la siguiente iteración (queda fuera de este cierre para no reabrir los bridges). En CRM y en el daemon el comportamiento es fail-closed y no presentan fallback.

3.2 Procedimiento de rotación de INTERNAL_API_SECRET

1. Generar el valor nuevo (openssl rand -hex 32) en el gestor de secretos del propietario; no pasar por chat ni por correo.

2. Editar el .env del host de producción (el compose lo comparte con app y wa-daemon) y las variables de servicio de WHATSPOWER y ULTRA si corren como procesos aparte en el mismo host.

3. Recrear en una sola ventana los procesos que lo consumen: docker compose up -d --force-recreate app wa-daemon, y reiniciar a continuación los servicios Node de WHATSPOWER y ULTRA (systemd restart).

4. Verificar por servicio: /api/doctor debe reportar longitudes de 64 caracteres; el daemon debe seguir 401 sin Bearer y 200 con el nuevo; una llamada firmada a auto-reply debe dar 200 y con el secreto viejo, 401; un dispatch de ULTRA contra el daemon debe pasar de 401 a 200.

5. Invalidar el valor antiguo en el gestor y registrar la fecha de rotación junto a las evidencias del capítulo 7.

Durante la ventana de recreación existen segundos en que app y daemon tienen valores distintos: las llamadas en vuelo reciben 401 y quedan registradas, sin pérdida de datos (el puente WHATSPOWER→CRM es fail-open controlado y reintenta en el siguiente evento). Por eso la rotación debe hacerse fuera de horas de validación y con las dos recreaciones encadenadas en el mismo comando.

3.3 APP_ENCRYPTION_KEY — matriz y procedimiento

La clave alimenta el módulo src/lib/crypto.ts (AES-256-GCM, clave derivada por SHA-256 del valor de entorno) y cifra en la base de datos los tokens de integraciones: telegramBotTokenEnc, instagramTokenEnc, googleRefreshTokenEnc, metaAppSecretEnc y whatsappCloudTokenEnc. El esquema local contiene además agentLeadsSecretEnc, pero ese campo pertenece al commit de AGENTE-LEADS que se excluyó de la publicación, así que la base de producción derivada de origin/main no lo tendrá. Si APP_ENCRYPTION_KEY no está definida, el módulo retrocede a APP_SECRET: en producción la variable debe fijarse siempre explícita para que ambos secretos sean independientes.

1. Generar clave nueva (openssl rand -hex 32) y conservar la antigua disponible solo durante la migración.

2. Si aún no existen integraciones guardadas (instalación reciente), basta con actualizar la variable y recrear app: no hay datos que re-cifrar.

3. Si ya hay tokens cifrados, ejecutar una pasada one-off que, con la clave vieja activa, lea cada campo con prefijo v1:, lo descifre con decryptSecret y lo reescriba cifrado con la nueva clave; verificar que ninguna fila conserva el prefijo con la clave vieja tras la pasada.

4. Comprobar funcionalmente: guardar un token nuevo en Configuración, recargar y confirmar que se lee y se enmascara correctamente; ejecutar la batería de crypto (tests/unit/crypto.test.ts) en el entorno.

5. Guardar la clave nueva junto a los backups: perderla equivale a perder los tokens cifrados y obliga a re-ingresarlos a mano (DEPLOY.md, sección 3).

4. Credenciales en artefactos locales e historial

Se ejecutó un escaneo final de credenciales sobre los artefactos locales del proyecto (repositorio CRM, clones frescos de WHATSPOWER y ULTRA en /home/z/repos, scripts, informes, logs y configuración git), con patrones de PATs de GitHub, JWT, claves albra_, claves NVIDIA, claves de proveedores de IA, claves AWS, bloques de clave privada y URLs de base de datos con contraseña. La pasada completa cubrió 1.476 archivos incluyendo plantillas internas de la plataforma; acotada al proyecto son 525 archivos, y el resultado es **cero credenciales reales**, con tres coincidencias triadas y descartadas de forma documentada. La evidencia completa quedó en download/evidencia-escaneo-credenciales.json, con valores enmascarados y motivo de descarte, sin conservar ningún valor sensible en el informe.

Coincidencia (enmascarada)	Ubicación	Motivo del descarte
albra_***ion (20 car.)	worklog.md:988	Nombre de directorio albra_specialization del repo AGENTE-GENERAL, no es una clave
albra_***000 (29 car.)	scripts/e2e-postpub-B.mjs:105	Fixture negativa de la prueba B7: clave inválida intencional para exigir 401
AKIAAA***AAA (20 car.)	plantilla de plataforma	Subcadena azarosa de un data-URI base64 de una imagen

Tabla 3. Falsos positivos del escaneo, triados con razón explícita.

Los artefactos e2e de fases anteriores mantienen su saneamiento: los seis archivos de token o clave (e2e-token.txt, e2e-wp-token.txt, e2e-apikey.txt, e2e-wp-key.txt, e2e-wp-apikey.json y e2e-demo-resp.json) contienen el marcador REDACTED desde el commit local de saneamiento b0af73f, que no está publicado, y los tres archivos de resultados no contienen secretos. La higiene git también está limpia: la URL de origin no lleva token embebido, no existe credential.helper ni ficheros .netrc o .git-credentials, y el commit con credenciales reales de pruebas (ab23795) y su informe (541d9b2) solo existen en ramas locales (local/sandbox-main-541d9b2 y main local), verificándose con git branch --contains que ninguna rama remota los contiene.

Respecto del historial ya publicado, dos secretos históricos siguen en main: la clave NVIDIA (nvapi-) y PATs antiguos en commits del worklog. Su eliminación del repositorio exige reescritura de historial (opcional, P2); su neutralización efectiva exige rotación y revocación, que forman parte de las acciones del propietario del capítulo 8. Las claves albra_ que usaron los artefactos e2e pertenecían a bases de datos ya destruidas y están redactadas, de modo que no conservan valor aprovechable.

5. Verificación de las ramas remotas publicadas

La verificación se hizo con git ls-remote anónimo contra los tres repositorios, sin usar ninguna credencial. Los tres SHAs publicados durante la misión coinciden byte a byte con lo que hoy expone origin/main; al tratarse de hashes de contenido, la igualdad de SHA implica igualdad criptográfica del contenido, de modo que no es necesario re-descargar y comparar archivos para afirmar que las ramas remotas contienen los commits correctos.

Repositorio	SHA publicado	SHA observado en origin/main	Resultado
ALBRA8/CRM-ALBRA	9b32591	9b32591aa73f...12354ab	**COINCIDE**
ALBRA8/WHATSPOWER	db0d088	db0d0883caca...3c44a0df	**COINCIDE**
ALBRA8/WHATSPOWER-ULTRA	08a0bb3	08a0bb398691...2f1f	**COINCIDE**

Tabla 4. Comparación SHA publicado contra origin/main (ls-remote anónimo, 2026-10-27).

En CRM-ALBRA existen además tres heads remotos ajenos a esta misión (backup-version-anterior, fix/whatsapp-production-media y snapshot-v2.3) que permanecen intactos, como en toda la operación: no hubo force push ni borrado de ramas. El contenido de cada commit de misión es exactamente el acordado: en CRM, 9b32591 añade cinco archivos (whatspower-bridge.ts de 170 líneas, su batería de 9 tests, inserciones en el webhook de WhatsApp y en opportunities/[id], y ajuste de tsconfig) sobre db95a5b; en WHATSPOWER, db0d088 toca cinco archivos (server.ts con HttpCrmClient real y el secreto dual AGENTIC_2A_WEBHOOK_SECRET sobre WEBHOOK_SECRET, router.ts con el fallback correlationId a eventId, types.ts, .env.example y una batería nueva de 3 tests de configuración de secretos); en ULTRA, 08a0bb3 corrige PNP-01 fijando businessDate para eliminar la dependencia del reloj en la prueba anti-ban.

Conviene recordar qué no se publicó a propósito: el commit 397418d (esquema AgentLeadEvent/agentLeadsSecretEnc para la futura fase AGENTE-LEADS), los commits de worklog y los scripts e2e con credenciales. Todo ello permanece solo en local, disponible para cuando se retome la fase LEADS, y su ausencia en origin/main es intencional y verificada.

6. Plan de despliegue a producción

El plan parte de la arquitectura ya documentada en DEPLOY.md (VPS con Docker, Caddy o Cloudflare Tunnel delante, app y daemon en la red interna) y la extiende con los dos servicios Node publicados en esta misión, que no tienen Dockerfile y deben correr como servicios de sistema en el mismo host. La regla de oro es arrancar de atrás hacia adelante: primero datos, luego CRM y daemon, después WHATSPOWER y al final ULTRA, que es el único capaz de originar envíos.

6.1 Variables y dependencias por servicio

Servicio	Puerto	Exposición	Comprobación de salud
Caddy / cloudflared	443	Único punto público (HTTPS)	curl -s https://dominio/api/health devuelve ok:true
CRM app (Next.js)	3000	Solo loopback 127.0.0.1:3000	/api/health ok:true y /api/doctor con longitudes 64
wa-daemon (Baileys)	3002	Interna (nunca pública)	GET sin Bearer responde 401 (vivo y protegiendo)
WHATSPOWER	3003	Interna	GET /api/health responde 200
WHATSPOWER-ULTRA	3004	Interna	GET de salud responde 200; ALLOW_REAL_DISPATCH=false

Tabla 5. Servicios, exposición de red y verificación de salud.

Servicio	Variables requeridas (nombres, nunca valores)
CRM app + wa-daemon (mismo .env)	DATABASE_URL, APP_SECRET, APP_ENCRYPTION_KEY, INTERNAL_API_SECRET (24+ car.), WHATSAPP_DAEMON_URL, WHATSAPP_VERIFY_TOKEN, SMTP_*; puentes: WHATSPOWER_ENABLED, WHATSPOWER_INBOUND_URL, WHATSPOWER_STAGE_URL, WHATSPOWER_WEBHOOK_SECRET; opcional ALBRA_AGENT_*
WHATSPOWER (.env propio)	PORT=3003, AGENTIC_2A_WEBHOOK_SECRET (debe igualar WHATSPOWER_WEBHOOK_SECRET del CRM), CRM_BASE_URL, CRM_API_KEY (clave albra_ de la organización creada en Configuración), WHATSAPP_DAEMON_URL, ALLOW_REAL_DISPATCH=false
WHATSPOWER-ULTRA (.env propio)	PORT=3004, CRM_BASE_URL, WHATSPOWER_BASE_URL, WA_DAEMON_URL, INTERNAL_API_SECRET (compartido), ALLOW_REAL_DISPATCH=false

Tabla 6. Variables por servicio; los tres secretos compartidos deben tener el mismo valor donde coincida el consumidor.

Dependencias previas: Docker + compose, dominio con registro A, puertos 22/80/443 abiertos (o solo 22 con túnel), IP estable del VPS y el repositorio clonado en /opt/crm-albra. Los contenedores corren como usuario no-root (uid 1001) y el arranque aplica el esquema con prisma db push fail-closed: un cambio destructivo detiene el contenedor en vez de perder datos. WHATSPOWER y ULTRA se registran como unidades systemd con WorkingDirectory en sus clones y ExecStart node --experimental-strip-types src/server.ts, heredando el .env del host mediante EnvironmentFile.

6.2 Orden de arranque y comprobaciones

1. Preparar .env con los secretos ya rotados (capítulo 3) y levantar BD y esquema: docker compose up -d --build; esperar app y wa-daemon en estado healthy.

2. Validar CRM: /api/health, /api/doctor (longitudes de secretos), registro del primer usuario owner y creación de la clave albra_ de la organización para WHATSPOWER.

3. Arrancar WHATSPOWER con AGENTIC_2A_WEBHOOK_SECRET igual al WHATSPOWER_WEBHOOK_SECRET del CRM y CRM_API_KEY recién creada; comprobar /api/health y una acción crm de prueba (get_clients) desde su log.

4. Arrancar ULTRA al final, con ALLOW_REAL_DISPATCH=false; comprobar salud y que cualquier intento de dispatch queda en DISPATCH_BLOCKED.

5. Registrar en el cuaderno de despliegue la salida de cada health check con fecha y SHA desplegado (evidencia P-01 y P-02 del capítulo 7).

6.3 Validación con WhatsApp controlado

La validación se hace con el número real de la empresa pero conversaciones 1-a-1 manuales con teléfonos del propio equipo, sin campañas, sin plantillas masivas y con ULTRA y WHATSPOWER en modo seguro. La batería replica la que pasó 25 veces o más en sandbox durante la fase post-publicación, de modo que los resultados esperados están calibrados:

- Entrada firmada: enviar un WhatsApp del equipo con la palabra de flujo; esperar cliente creado con source=crm-api y nota en el historial; repetir el mismo evento y confirmar deduplicated (idempotencia).
- Firma y replay: reenviar un webhook con firma inválida debe dar 401; reenviar uno con timestamp antiguo debe ser rechazado por anti-replay; un payload incompleto, 400.
- Cambio de etapa: mover una oportunidad a PROPUESTA_ENVIADA y verificar que el puente notifica a WHATSPOWER y el trigger queda persistido sin envío real.
- Aislamiento multi-tenant: intentar desde la organización B operar sobre un cliente de la organización A; debe ser bloqueado (401/denied).
- Resiliencia: detener el CRM un instante y comprobar que WHATSPOWER responde 200 con crmSynced=false (fail-open controlado) y que al volver, el siguiente evento sincroniza.
- Guard de difusión: intentar un dispatch con ULTRA y verificar DISPATCH_BLOCKED mientras ALLOW_REAL_DISPATCH=false.

6.4 Autenticación, persistencia y recuperación

Autenticación: sesión JWT con APP_SECRET, claves API albra_ por organización para el puente crm/actions, Bearer compartido entre app, daemon, WHATSPOWER y ULTRA, y firmas HMAC en el webhook de Meta y en el puente CRM→WHATSPOWER. Persistencia: SQLite en volumen db-data (custom.db) y sesión de WhatsApp en volumen wa-auth, de modo que reiniciar el VPS no pide re-escanear el QR; solo se re-escanea si se cierra sesión desde el teléfono o se borra el volumen. Recuperación: cron nocturno con backup-nightly.sh a las 03:30 (snapshot consistente de SQLite con integrity_check, empaquetado del volumen wa-auth, retención de 14 días y envío externo con rclone si se define BACKUP_REMOTE_DEST).

Antes de dar por bueno el despliegue debe ejecutarse el ensayo de desastre: bash scripts/backup-restore-drill.sh debe terminar en exit 0 (PASS), y conviene completar el drill manual que además restaura el volumen wa-auth siguiendo el procedimiento de la sección 8 de DEPLOY.md, sin olvidar el chown 1001:1001 de los volúmenes restaurados. Este ensayo produce la evidencia P-07 del capítulo 7.

6.5 Rollback

- Código: anotar el SHA desplegado; para revertir, git checkout en el repo afectado y reconstruir (docker compose up -d --build en CRM; reinicio del servicio systemd en WHATSPOWER y ULTRA).
- Base de datos: restaurar el snapshot nocturno previo al cambio con el procedimiento de DEPLOY.md sección 8 (stop, copiar custom-FECHA.db, limpiar -wal/-shm, chown 1001:1001, up).
- Secretos: mantener el valor anterior en el gestor durante toda la ventana; si la rotación de APP_ENCRYPTION_KEY hay que deshacerla, re-ejecutar la pasada de re-cifrado en sentido inverso con la clave vieja recuperada.
- Funcional: los interruptores ALLOW_REAL_DISPATCH=false y WHATSPOWER_ENABLED permiten aislar el comportamiento de los puentes sin revertir código.

7. Criterios para declarar "Producción Verificada"

Nada de lo verificado hasta la fecha —ni los 25+ tests funcionales de sandbox ni la verificación remota de este cierre— constituye verificación de producción. El dictamen solo puede cambiarse cuando los ocho criterios siguientes se hayan ejecutado en el entorno real y su evidencia quede archivada. Cada criterio es binario: se cumple o no, y la evidencia requerida es explícita para que no quepa una declaración ambigua.

ID	Criterio	Evidencia requerida
P-01	Stack desplegado y sano en el VPS	docker compose ps con app y wa-daemon healthy; /api/health ok:true
P-02	Perímetro correcto	HTTPS activo por dominio; 3000 publicado solo en loopback; 3002/3003/3004 no alcanzables desde Internet
P-03	Secretos nuevos en producción	INTERNAL_API_SECRET y APP_ENCRYPTION_KEY rotados según capítulo 3; PAT expuesto devolviendo 401; /api/doctor con longitudes 64
P-04	Sesión WhatsApp operativa y persistente	QR escaneado una vez; reinicio del daemon sin re-escaneo; mensaje entrante y saliente de prueba con teléfonos del equipo
P-05	Batería funcional A/B/C en verde sobre producción	Flujo firmado crea cliente (source=crm-api); stage-change dispara trigger sin envío real; idempotencia deduplicated; 401 firma/replay; multi-tenant bloqueado; fail-open con CRM caído y recuperación
P-06	Guard de difusión activo	ALLOW_REAL_DISPATCH=false verificado en WHATSPOWER y ULTRA; intento de dispatch termina en DISPATCH_BLOCKED
P-07	Copias de seguridad y recuperación probadas	Primer backup nocturno generado; backup-restore-drill.sh exit 0 en el VPS; copia externa verificada
P-08	Evidencia documentada y aceptada	Cuaderno de despliegue con fechas, SHAs, salidas de salud y resultados P-01..P-07; conformidad escrita del propietario

Tabla 7. Criterios binarios de aceptación para el dictamen de Producción Verificada.

Solo con los ocho criterios en verde el informe de fase podrá afirmar que el sistema está verificado en producción. Si algún criterio falla, se documenta el fallo con su evidencia, se corrige y se re-ejecuta el criterio completo, sin aprobaciones parciales.

8. Acciones del propietario y anexo operativo

Las siguientes acciones requieren intervención exclusiva del propietario; ninguna es ejecutable desde esta sesión sin reutilizar credenciales expuestas o sin autorización expresa. Están ordenadas por prioridad real: las tres primeras bloquean el despliegue, las siguientes lo acompañan.

Pri.	Acción	Detalle
1	Revocar el PAT expuesto y los PATs antiguos	github.com/settings/tokens; verificación 401 fuera de chats (capítulo 2)
2	Rotar la clave NVIDIA	console.ngc.nvidia.com: revocar la nvapi- del historial publicado y crear una nueva en Configuración → Agente IA
3	Rotar INTERNAL_API_SECRET y APP_ENCRYPTION_KEY	Procedimiento y verificación por servicio del capítulo 3; fijar siempre APP_ENCRYPTION_KEY explícita
4	Aprobar (o no) el cambio del fallback de WHATSPOWER	Eliminar el valor por defecto de INTERNAL_API_SECRET en src/server.ts es un cambio de código para la próxima iteración

Pri.	Acción	Detalle
5	Autorización expresa escrita para campañas reales	Sin ese documento, WHATSPOWER y ULTRA permanecen con ALLOW_REAL_DISPATCH=false
6	Ejecutar el plan del capítulo 6 y registrar P-01..P-08	Cuaderno de despliegue con evidencias; dictamen final en una nueva fase

Tabla 8. Acciones pendientes del propietario, en orden de prioridad.

Anexo: verificación rápida post-despliegue

Los comandos siguientes resumen las comprobaciones de salud y de seguridad del plan y pueden copiarse tal cual en el VPS tras cada arranque o actualización. Devuelven estados y longitudes, nunca valores de secretos.

```
# Salud del stack
docker compose ps
curl -s http://127.0.0.1:3000/api/health
curl -s http://127.0.0.1:3000/api/doctor | jq .
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:3002/ # 401 esperado
curl -s http://127.0.0.1:3003/api/health

# Perimetro: nada de esto debe ser alcanzable desde Internet
curl -s -o /dev/null -w "%{http_code}\n" http://<IP-publica>:3000/ # debe fallar

# Estado de los secretos (longitudes, sin valores)
grep -c "=" /opt/crm-albra/.env # inventario, no valores

# Guard de difusion (debe quedar bloqueado)
ALLOW_REAL_DISPATCH=false # en .env de WHATSPOWER y ULTRA

# Evidencia y rollback
git -C /opt/crm-albra rev-parse HEAD
bash /opt/crm-albra/scripts/backup-restore-drill.sh # exit 0 = PASS
```

Con las acciones 1 a 3 completadas y el plan del capítulo 6 ejecutado con sus evidencias, el proyecto quedará en condiciones de recibir el dictamen de Producción Verificada en una siguiente fase de validación, manteniendo intactas las restricciones de no campañas y no mensajería real sin autorización expresa.